

National College of Ireland
Project Submission Sheet



Name	Chethan Mundigehalla Prabhakar
Student-id	X23297395
Module	Cloud Platform Programming
Year	2024
Course	Cloud Computing
Project Title	TuneStream with AWS
Submission Date	02-12-2024
Due Date	02-12-2024
Word Count	3986

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard supplied by the library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Signature	Chethan Mundigehalla Prabhakar
Date	02-12-2024

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. You must ensure that you retain a **HARD COPY** of ALL projects, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. Late submissions will incur penalties.
5. All projects must be submitted and passed in order to successfully complete the year. Any project/assignment not submitted will be marked as a **failure**.

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement
Cloud Platform Programming
TuneStream With AWS (Amazon Web Services) Resources

Your Name/Student Number	Course	Date
Chethan Mundigehalla Prabhakar	MSc in Cloud Computing	02-12-24

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click [here](#).

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
NA	NA	NA

TuneStream With AWS (Amazon Web Services) Resources

<http://production1.eba-iq7mhzsh.us-east-1.elasticbeanstalk.com/login?next=/>

Chethan Mundigehalla Prabhakar

Student-Id: 23297395

Cloud Platform Programming, MSc in Cloud Computing

National College of Ireland Dublin, IRELAND

Email: x23297395@student.ncirl.ie. URL: www.ncirl.ie

Abstract

Tunestream — A web based music application with seamless experience by AWS (Amazon web services) 7 AWS services which include Amazon RDS as the database, Cloud9 and the deployment using Lambda, SNS (Simple Notification Service), EC2, S3 (Simple Storage Service) and EBS (Elastic Bean stalk). The Tunestream app then has a simple user interface for login, signup, view, play and pause, song of all 7 top artist and top songs. We used Rapid API for getting all of the songs and the top artists for the app. Best user friendly xanax cloud based solution in entertainment industry. BackEnd: Python Django Frontend: HTML and Tailwind CSS Version Control System: GitHub private repository The app uses Python Django as backend framework, HTML and Tailwind CSS as frontend where GitHub private repository is used as version control system. The purpose of the application is to provide a users with an error-free listening experience of each song with the best audio quality in a secure environment. This amazing project helps us to work in technologies like Django, HTML, Tailwind CSS and various AWS services as well more importantly helps us in learning the enormous depth involved in building web applications.

Introduction

Cloud computing has disrupted the entertainment industry in the digital era, providing applications that are scalable, efficient, and user-friendly. Tunestream is a cutting-edge cloud music streaming application that offers a streamlined and enjoyable experience for its users. The application is built on top of Amazon Web Services (AWS), taking full advantage of a powerful, scalable, fault-tolerant cloud infrastructure. The Tunestream architecture harnesses the power of seven AWS Services: Amazon RDS for database management, Cloud9 for development, Lambda for serverless computing, SNS (Simple Notification Service) for notifications, EC2 as compute capacity, S3 as storage, and Elastic Beanstalk for deployment. A friendly and accessible interface allows users to sign up, log in, explore songs and artists, and interact with music. It uses the content from Rapid API and showcases top artists and songs for making the process

easier. Tunestream is a modern and energetic solution of the entertainment industry of Europe developed with Python Django as a backend framework and Tailwind CSS for the frontend

Functional and Non-Functional Requirements

A detailed overview of functional and non-functional requirements needs to be there when building a web application in my case it is a music base web application known as TuneStream. The app uses powerful features of AWS such as RDS, SNS, S3, clou9, Elastic Beanstalk and lambda.

Functional requirements

when a user enter the website he will see a beautiful login screen from which he can login or if he is not a registered user he can use the signup button to register and login securely. Users can browse all the songs just by scrolling or by search option which is present. The user can click on the song which he likes and this will take him to profile page of the song where he can play the song and pause it whenever he likes he can also click on forward arrow button to forward the song for 10 seconds and backward arrow to reverse the song for 10 seconds. There is also a subscription button from which user can give his card details such as card number, Expiry date and CVV and become a subscribed user by clicking on pay button. If the user wishes to unsubscribe he can click on unsubscribe button to get successfully unsubscribed (CRUD Operations). There is a search button as well for seamless experience of user from which he can search the artists present or search for the songs which he might like. The admin will get the alert once the user registers where he can see the user name and his mail information which is achieved by using a AWS service known as SNS (Simple Notification Service).

Non-Functional Requirements,

Tunestream is able to handle growing user base with AWS Elastic Beanstalk and EC2 for resource management which helps in scalability. Tune stream will make sure that the data of customer is safe by using AWS services which is most reliable in terms of

there is any changes in this bucket like update or delete operations the admin will be notified using Amazon SNS service.

The architecture of Tunestream Web-application, this web app is designed to leverage benefits offered by several AWS services to provide scalability, performance, reliability, security cost efficiency and a better user experience. The web app itself has two main parts backend and frontend. Webapp is built with python Django as its backend framework, HTML and CSS as its frontend, and uses AWS resources via python. Once the user enter the application he will see the login page if he is a registered user and directly login to the application or else the user should click on signup button and register to the app. Once the user registers the admin will receive a notification message saying the user with this username and mail-id has registered successfully this will use the AWS SNS(Simple notification Service) service to send the notifications to the admin. All the login and Subscription information of user are securely store in a Database Amazon RDS PostgreSQL. Once user enters the application he will be able to see the Top Artists and Top songs he can listen to songs whichever he likes, can search the songs and play, pause and fast-forward the songs as well. All the song, artist and user profile information is fetched from the RAPID API It simplifies the integration of APIs into applications by offering a unified interface and tools for API testing, monitoring, and scaling.. The application is deployed using AWS Elastic Beanstalk which will fetch its requirement from the S3 bucket every time we deploy a application a ZIP file is stored in a S3 bucket and use EC2 instance resources to deploy the application. Every time we upload or delete an file in S3 bucket it will trigger the AWS lambda function which send a mail to the admin saying that a file is uploaded or deleted using Amazon SNS service. There is also a Amazon S3 bucket which is created programmatically where all your project is uploaded for backup purpose even if

Fig: TuneStream Architecture

The TuneStream web application makes use of 7 AWS services. S3, RDS, SNS, Lambda, Elastic Beanstalk, EC2 and cloud9.

Advantage and Disadvantage of Each services used.

1) **Amazon S3 (Simple storage Service):** AWS offers suites of services to store data and manage information. Amazon Simple Storage Service (S3) is the flagship offering in this domain. You can store information like data objects, customized shelves, databases, etc. S3's primitives are two: bucket and object. Objects are what is actually stored, and buckets are virtual containers in which to place the objects. Metadata can also be added to objects to metadata enrich an object — to tag the stored content with additional information. [1]

Pay-as-you-go pricing with different storage classes such as S3 Standard, S3 Glacier, etc Works out of the box with other AWS services such as Lambda, EC2 and CloudFront These security features include bucket policies, encryption, and access control. [2]

Disadvantages

Frequent access, requests, and data transfer can increase your costs dramatically. Not ideal for use cases needing low-latency block storage Not to mention it is probably not the best overall performance for small files. [2]

For my TuneStream web application I have chosen Amazon S3 as a backup solution where I uploaded all my application information in a zip file and kept and even the reports are present inside this. Security is the main priority of Tunestream keeps its data and deployment by the use of S3 bucket

2)RDS: Relation Database Service:

Advantages

Management Made Easy: Handles database provisioning, patching, backups, and scaling automatically. High Availability: Multi-AZ deployments help to enhance database availability. Multiple Engines Supported: MySQL, PostgreSQL, Oracle, SQL Server, and other. Performance Tuning: Provides features such as read replicas for performance enhancement. [3]

Disadvantages

Cost: It can be costly for scale databases or high IOPS. Limited Access to Underlying OS. Users are not in full control of configuring the database as extensively as they can do on the OS level. [4]

In Tunestream we have used PostgreSQL service for database which is option available from RDS.

Tunestream uses this service to store user information such as email-id, password, username etc.. and it stores the information about the subscription of the user. Tunestream uses this service as provides high performance and keeps the

data safe by using features such as VPC and network isolation.

3)SNS: Simple Notification Service: Amazon SNS uses a publish-subscribe mechanism which connects heterogeneous applications. Unlike Amazon SQS — where you need to keep polling a certain queue to see a new message that needs to be processed — with Amazon SNS your applications can be notified when new relevant content is available. [5]

Advantages

Scalability: Millions of messages per day. Flexibility: You can use multiple delivery formats (email, SMS, HTTP/HTTPS, and AWS Lambda). Reliability — Built-in retries and fault tolerance for message delivery. Simple Integrations Integrates with AWS services like S3, CloudWatch, and Lambda. [6]

Disadvantages

High Cost: Sending SMS can either be quite expensive depending on the region and volume. Then Limit Size & Type: When customizing messages. No Message Queuing: Lacks native support for the message queuing and therefore requires integration with services like Amazon SQS. [7]

TuneStream uses this service to notify the admin, whenever a user gets registered the admin will get notification which says user with username and mail-id registered successfully. It will also notify the admin whenever anyone uploads or delete a file from s3 bucket as a kind of security purpose. The main reason for using this service is that it supports multiple endpoint such as mail, SMS etc to reach out and encryption methods to send notifications securely.

4)Lambda: A serverless service which responds to events occur in AWS.

Advantages

No Infrastructure Management: All servers are managed at abstract level.

Elasticity : Auto scale as the workload requires. Integration: Easily integrates with AWS services such as S3, DynamoDB and SNS. [8]

Disadvantages

Cold starts: that lead to latency on the first invocation while initializing the container.

Execution Time Limits: 15 minutes per invocation.

Windows porting: As microservices are developed in a wider range of programming languages than traditional architectures, porting it to Windows is sometimes more difficult.

State Management: Stateless by nature, needs external services to retain state. [9]

In TuneStream we have used lambda function to trigger the bucket update and deletion of S3 bucket will trigger the lambda function and will send the notification for admin using AMAZON SNS services. This service provides serverless computing and no management is required the main reason for being used in our application.

5)Elastic Beanstalk: AWS Elastic Beanstalk is the simplest way to get applications up and running on the AWS cloud. This service makes it easier to provision instances, deploy application code, and grant access to them. This service is presently available only for Java/Tomcat Web applications technology stack. Developers have the option to easily bundle Web apps into a WAR file and use Beanstalk for automated deployment to the AWS Cloud.[10]

Advantages

Flexibility: Supports multiple programming languages (Java, Python, PHP, etc.)

Auto Scale: Automatically scales according to application load.

Integration: Integrates well with other AWS services, such as RDS and S3. [11]

Disadvantages

Cost: There could be higher costs of managed features over the manual setups.

Tune stream uses Elastic beanstalk for deployment. It auto scales itself whenever necessary. [11]

Library Description and Creation

Generally, a library is a collection of books or a hall or a location where numerous books have been kept for in future. Similarly a library in programming is a collection of some precompiled codes to be used later in the program for some well-defined specific operations. A library can also contain documentation, configuration data, message templates, dependencies, classes, values, and many other things, aside from compiled codes. Essentially, a Python library is a collection of modules. It comes with packages of code that you can reference in different programs. Consider Python because we don't have to write the same code for different programs Python Libraries — Machine Learning, Data Science & Data Visualization Python libraries are really important and are being used for Machine Learning, Data Science, Data Visualisation etc Tunestream depends on a library named subscription_libnci which is available in PyPI Python Package Index (PyPI)

Overview

1)Django's User model: Represents application users

2)Subscription model:

Representation of subscription data per user, with this fields.

user: ForeignKey or OneToOneField linked to User.

is_subscribed: Whether user is subscribed.

3)Transaction Management: It guarantees atomicity of the database operations to ensure consistency.

```
from django.contrib.auth.models import User
from django.db import transaction
from music.models import Subscription
```

Functionality

1)subscribe_user(user): When a user is not already subscribed, this function subscribes him/her.

Steps:

Get or create a user Subscription: Subscription.objects.get_or_create(user=user) guarantees that there is always a Subscription object for the user.

If the subscription already exists created is False.

Check if the user is already subscribed: If subscription.is_subscribed == True, do not Do save (0 means gethere)

Otherwise: If you are using transaction block (transaction.atomic()) to make sure any changes to the database are done atomically. Update is_subscribed to True.

Save the subscription. Returns True to indicate success of the subscription.

Benefits of get_or_create:

Avoids duplicate subscriptions being entered.

Thread-safe, for if two processes try to create subscriptions at the same time.

```
def subscribe_user(user):
    """Subscribes a user if they are not already subscribed."""
    # Use get_or_create to avoid duplicate subscriptions
    subscription, created = Subscription.objects.get_or_create(user=user)

    if subscription.is_subscribed:
        return False # Already subscribed, so return False

    with transaction.atomic():
        subscription.is_subscribed = True
        subscription.save()
    return True # Successfully subscribed
```

2)unsubscribe_user(user) : This part of the code will unsubscribe the user if it is subscribed.

Steps:

Query for a subscription where user matches and is_subscribed is True:

Subscription.objects.filter(user=user, is_subscribed=True).first() retrieves the subscription if it exists.

If no subscription exists, return False (no action taken).

Otherwise:

Set is_subscribed to False.

Save the subscription to update the database.

Return True to indicate the user was successfully unsubscribed.

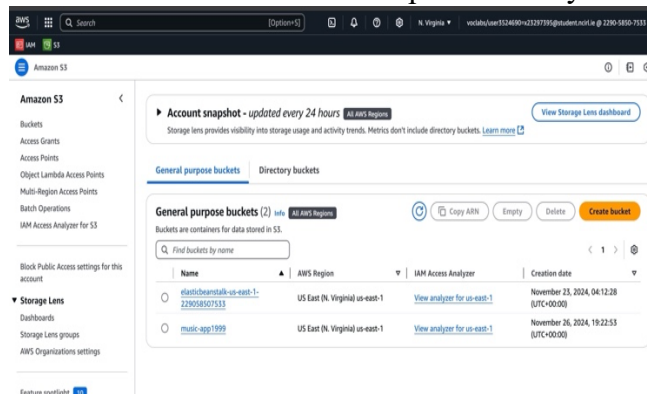
```
def unsubscribe_user(user):
    """Unsubscribes a user if they are subscribed."""
    subscription = Subscription.objects.filter(user=user, is_subscribed=True).first()
    if subscription:
        subscription.is_subscribed = False
        subscription.save()
        return True
    return False
```

You can download this package by using the link i.e. [subscription_libnci-0.1-py3-none-any.whl](#) or by entering the command `pip install subscription_libnci`. [12]

Implementation

The Tunestream application makes use of 6 AWS services such as S3,Elasticbeanstalk,lambda,SNS,RDS, cloud9. Each service has its own usage and its implementation is demonstrated in this section.

1)S3 : I have created s3 bucket the name of the bucket is music-app1999 which is created programmatically. Tunestream uses this bucket to keep backup files. There is another bucket which is created by elastic beanstalk for keeping the deployed zip files and other required. A attached screenshot of buckets present in my account.



Attached programmatically creation of music-app1999 bucket.

```
0e7d0b6f474af19122ec17460e2c34?region=us-east-1
Welcome x s3event.py x s3.py
1 import boto3
2 from botocore.exceptions import ClientError
3
4 def create_bucket(bucket_name, region='us-east-1'):
5     """Create an S3 bucket with the given name and region (default: us-east-1)."""
6     try:
7         # Initialize the S3 client with the specified region
8         s3_client = boto3.client('s3', region_name=region)
9
10        # Create the bucket with a location constraint (for regions other than us-east-1)
11        if region == 'us-east-1': # No location constraint needed for us-east-1
12            s3_client.create_bucket(Bucket=bucket_name)
13        else:
14            s3_client.create_bucket(
15                Bucket=bucket_name,
16                CreateBucketConfiguration={'LocationConstraint': region})
17
18        print(f"Bucket '{bucket_name}' created successfully in region '{region}'.")
19        return True
20    except ClientError as e:
21        print(f"Error occurred: {e}")
22        return False
23
24
25 if __name__ == "__main__":
26     bucket_name = "music-app1999" # Bucket name must be globally unique
27     region = "us-east-1" # You can change this to any AWS region (e.g., 'us-west-2', 'eu-west-3')
28     create_bucket(bucket_name, region)
29
```

2)Elastic Beanstalk: Tunestream is deployed using elastic beanstalk service present in AWS. The elastic bean stalk service uses requirements.txt file to fetch all the details which is required to run the application. Once you deploy a application in elastic beanstalk it will automatically create a S3 bucket, EC2 instance, load balancing, autoscaling etc.. Elastic beanstalk uses S3 bucket to store all the deployed files in it and it will uses the resources and get the IP from an deployed EC2

instance. Before deploying a Elastic beanstalk you need to create folder name .ebextensions where you will create django.config file and mention the name of the project and location for your static files.

Commands used while creating an EC2 instance:

eb init :

This will Sets up the configuration files required to deploy an application to Elastic Beanstalk. Creates the .elasticbeanstalk directory in your project, which contains configuration files like config.yml and credentials. It will also prompts you to mention the version of python you are using and the location which you need the application to be deployed.

eb create environment-name --service-role name --instance_profile name:

eb create is command used to create the environment in -name section you need to provide the environment name i.e. production1 in my case --service-role stands for name of the IAM role you are using to deploy the application, as I don't have permission to create new role I have used the existing role i.e. Labrole and the -instance_profile represents profile name of the IAM user Labrole I have used the existing instance profile name for labrole.

eb deploy

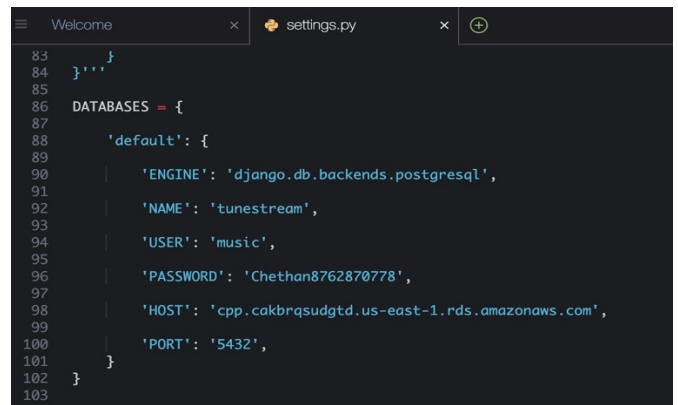
eb deploy will deploy the application in an ec2 instance and create an URL for accessing the application.

3)RDS: Tunestream uses the RDS PostgreSQL database

as the database for this implementation I have commented out the default database connection made from Django to SQLite and I have included the AWS database connection lines. Before making the migration I have removed the previous migration file and did the migration to new database.

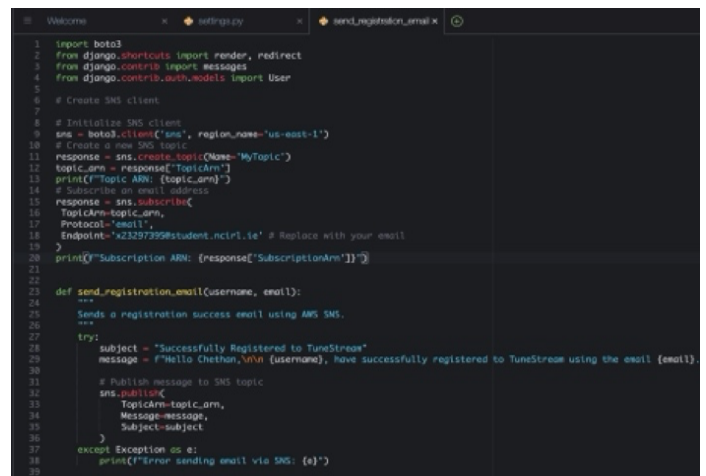
- Python3 manage.py makemigrations
- Python3 manage.py migrate

Screenshot for settings.py for making connection with RDS PostgreSQL is attached below.



```
83 }
84 }
85
86 DATABASES = {
87     'default': {
88         'ENGINE': 'django.db.backends.postgresql',
89         'NAME': 'tunestream',
90         'USER': 'music',
91         'PASSWORD': 'Chethan8762870778',
92         'HOST': 'cpp.cakbrqsudgtd.us-east-1.rds.amazonaws.com',
93         'PORT': '5432',
94     }
95 }
```

4)SNS: Tunestream uses SNS as a notification service. The admin will receive the notification if any new user is registered in the application. For this implementation I have used the following code attached in the screenshot.

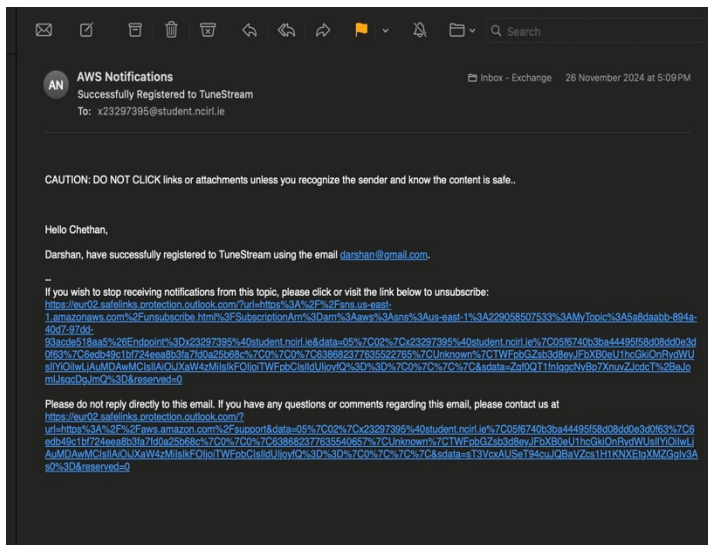


```
1 import boto3
2 from django.shortcuts import render, redirect
3 from django.contrib import messages
4 from django.contrib.auth.models import User
5
6 # Create SNS client
7
8 # Initialize SNS client
9 sns = boto3.client('sns', region_name='us-east-1')
10 # Create a new SNS topic
11 response = sns.create_topic(Name='MyTopic')
12 topic_arn = response['TopicArn']
13 print(f"Topic ARN: {topic_arn}")
14 # Subscribe an email address
15 response = sns.subscribe(
16     TopicArn=topic_arn,
17     Protocol='email',
18     Endpoint='x23297395@student.ncirl.ie' # Replace with your email
19 )
20 print(f"Subscription ARN: {response['SubscriptionArn']}")
21
22
23 def send_registration_email(username, email):
24     """
25     Sends a registration success email using AWS SNS.
26     """
27     try:
28         subject = "Successfully Registered to TuneStream"
29         message = f"Hello Chethan, you have successfully registered to TuneStream using the email {email}."
30         # Publish message to SNS topic
31         sns.publish(
32             TopicArn=topic_arn,
33             Message=message,
34             Subject=subject
35         )
36     except Exception as e:
37         print(f"Error sending email via SNS: {e}")
38
39
```

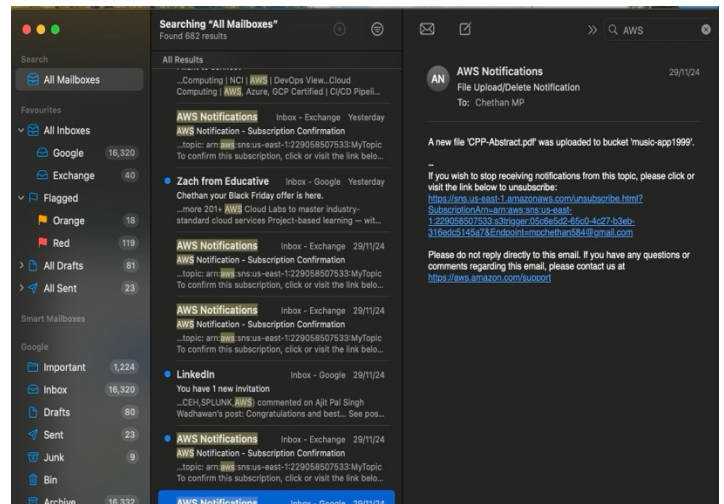
The part of the code in the screenshot create a SNS topic named MyTopic and creates a subscription with the mail-id x23297395@student.ncirl.ie.

Every time a new user is registered there will be a notification in the mail-id mentioned.

Attached screenshot below of a mail received.



there is any modification in the S3 the above mail-id will be receiving the notification.
Attached screenshot of a mail received.



5)Lambda: Tunestream uses AWS Lambda function to trigger if anything gets uploaded or deleted from S3 bucket. AWS lambda uses SNS to send notifications to admin, The name of the lambda function is S3-invoke which is created by using Labrole. I have created a trigger for both s3 buckets where I selected both put object and delete object function for both buckets. For destination we have used SNS topic asynchronous invocation. Here is the screenshot attached. In the below code we have mentioned what message the admin should receive if buckets are modified and the SNS topic ARN to which subscription this message should go.

```

9      try:
10         for record in records:
11             # Process the message only based on event type
12             if event_name.startswith('ObjectCreated'):
13                 # File was uploaded
14                 message = f"A new file '{object_key}' was uploaded to bucket '{bucket_name}'."
15             elif event_name.startswith('ObjectRemoved'):
16                 # File was deleted
17                 message = f"The file '{object_key}' was deleted from bucket '{bucket_name}'."
18             else:
19                 # If the event type is unknown or not handled, skip it
20                 continue
21
22             # Send an SNS notification
23             sns_client.publish(
24                 TopicArn=sns_topic_arn,
25                 Subject='File Upload/Delete Notification',
26                 Message=message
27             )
28
29         return {
30             'statusCode': 200,
31             'body': json.dumps('Event processed successfully')
32         }
33     except Exception as e:
34         print(f"Error processing event: {str(e)}")
35         return {
36             'statusCode': 500,
37             'body': json.dumps('An error occurred while processing the event')
38         }

```

Have created a SNS topic S3 trigger and the subscription mail-id is mpchethan584@gmail.com so if

Conclusion

Tunestream is a contemporary, cloud-oriented solution that utilizes the capabilities of AWS and modern web technologies to provide an effortless way to stream music. This application has a heavy backend (Python Django) and a responsive frontend (HTML + Tailwind CSS) that provides the user with tons of features and a good interface. With Rapid API integration, it can guarantee access to up-to-date, diverse and music library of top artists and songs.

Tunestream is designed to scale, be reliable and performant by leveraging seven services, Amazon RDS to manage the database, Cloud9 for the development environment, Lambda for backend triggers, SNS to send notifications, EC2 for computing, S3 for secure storage, and Elastic Beanstalk for deployment. Overall, Tunestream is a well-designed and strategic service concept that successfully uses the latest in cloud technology with a strong focus on the user experience to meet the rising demand for music streaming services. The creative structure and design of this solution guarantees its viability and flexibility in the entertainment field.

I have faced many challenges while creating this application while deploying the application I have a misconfiguration in my requirements.txt file from

which I was getting a bad gateway error even after deploying the application. I have tried to link s3 bucket created to static files in Django the URL was updating but I was not access any images of application publicly as URL was not public. Then I started using that bucket as a backup bucket. While creating a lambda function the mail was not triggered even when I upload or delete a folder in S3 bucket this was resolved by adding the proper TopicArn and making some changes in the program.

In future if I get a chance to implement this project again I will create a music library for each user, I will also try and implement MFA by integrating Forti MFA with AWS. I will try to integrate payment gateway with the application so he can subscribe successfully to the app. For managing high user traffic effectively, use AWS Elastic Load balancer to evenly distribute load across servers. Ensured seamless updates and delivery of new features faster by automation into continuous integration and deployment by use of AWS code deploy, AWS code commit and code build Integrate third-party APIs that allow users to publish their playlists and favorite songs onto social media platforms.

Running application Link

<http://production1.eba-iq7mh2sh.us-east-1.elasticbeanstalk.com/login?next=/>

GitHub Private Repository link

https://github.com/ChethanNCI/spotify_clone.git

PyPI Library Download Link:

[subscription_libnci-0.1-py3-none-any.whl](#)

References

- [1] S. S. V. Buyya Rajkumar, "NCI Library," [Online]. Available:] <https://r3.vlreader.com/EpubReader?ean=1780124095397#>. [Accessed 30 November 2024].
- [2] Amazon, Amazon, [Online]. Available:] <https://aws.amazon.com/s3/>. [Accessed 28 November 2024].
- [3] "Amazon," Amazon, [Online]. Available:] <https://aws.amazon.com/rds/features/>. [Accessed 28 November 2024].
- [4] Rahul AwatiBen Lutkevich. [Online]. Available:] <https://www.techtarget.com/searchaws/definition/Amazon-Relational-Database-Service-RDS>. [Accessed 29 November 2024].
- [5] C. V. S. T. S. Rajkumar Buyya, "NCI Library," in *Mastering Cloud Computing Foundations and Applications Programming*, Morgan Kaufmann, pp. 201-203.
- [6] "Amazon," [Online]. Available:] <https://aws.amazon.com/sns/features/>. [Accessed 29 November 2024].
- [7] W. Silva. [Online]. Available:] https://medium.com/@vava_silva/understanding-aws-sqs-and-sns-common-questions-best-use-cases-and-pros-cons-fd5015bead77. [Accessed 30 November 2024].
- [8] "Amazon," Amazon, [Online]. Available:] <https://aws.amazon.com/lambda/>. [Accessed 30 November 2024].
- [9] G. C. David Jackson. [Online]. Available:] <https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1028&context=ittscicon>. [Accessed 30 November 2024].
- [1] Rajkumar Buyya, Chirstian Vecchiola, 0] "https://r4.vlreader.com/EpubReader?ean=1780124095397#," in *Mastering Cloud Computing: Foundation and Application*, Morgan Kaufmann, pp. 195-196.
- [1] A. Camp. [Online]. Available: 1] <https://medium.com/@acamp/elastic-beanstalk-advantages-and-drawbacks-be814615af01>. [Accessed 30 November 2024].
- [1] D. S. Jaiswal. [Online]. Available: 2] https://moodle2024.ncirl.ie/pluginfile.php/44829/mod_resource/content/2/Tutorial_Creating_a_Python_Library.pdf. [Accessed 30 November 2024].